

Docker and Kubernetes



Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

Session 1

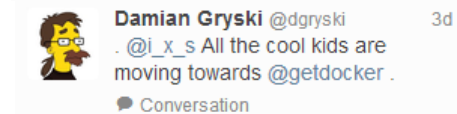
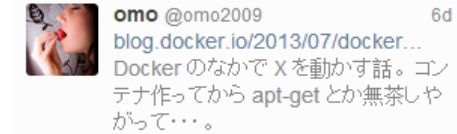
Getting started with Docker

Agenda

- Introduction
- Docker Overview
- Getting Started with Docker
- Basic commands

In the 8 months since we launched

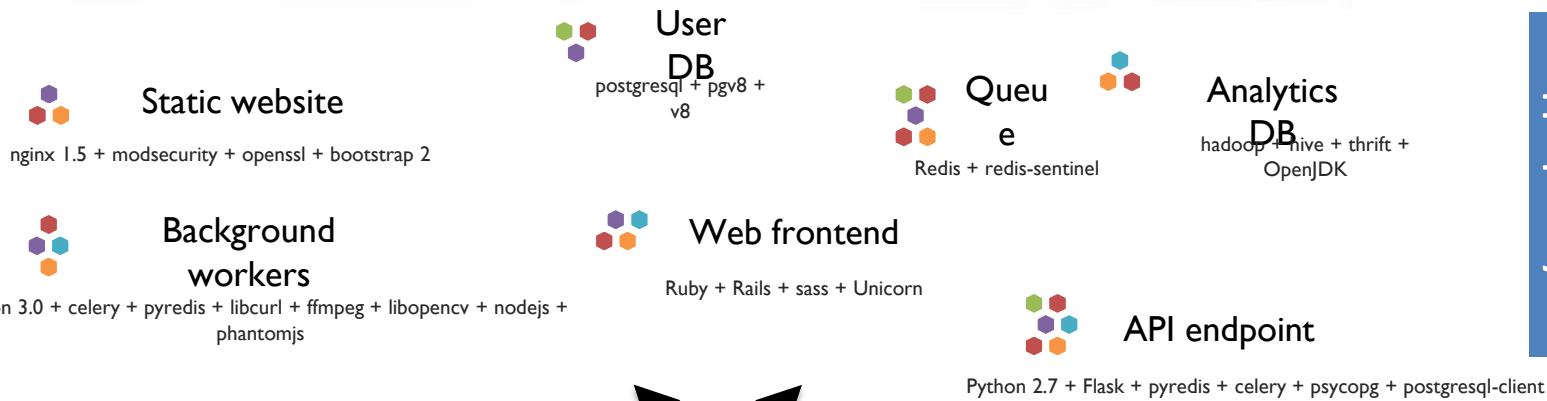
- >200,000 pulls
- >7,500 github stars
- >200 significant contributors
- >200 projects built on top of docker
 - ✓ UIs, mini-PaaS, Remote Desktop...
- 1000's of Dockerized applications
 - ✓ Memcached, Redis, Node.js...and Hadoop
- Integration in Jenkins, Travis, Chef, Puppet, Vagrant and OpenStack
- Meetups arranged around the world...with organizations like Ebay, Cloudflare, Yandex, and Rackspace presenting on their use of Docker



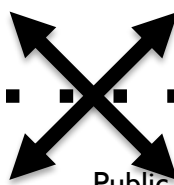
WHY ALL THE EXCITEMENT?

The Challenge

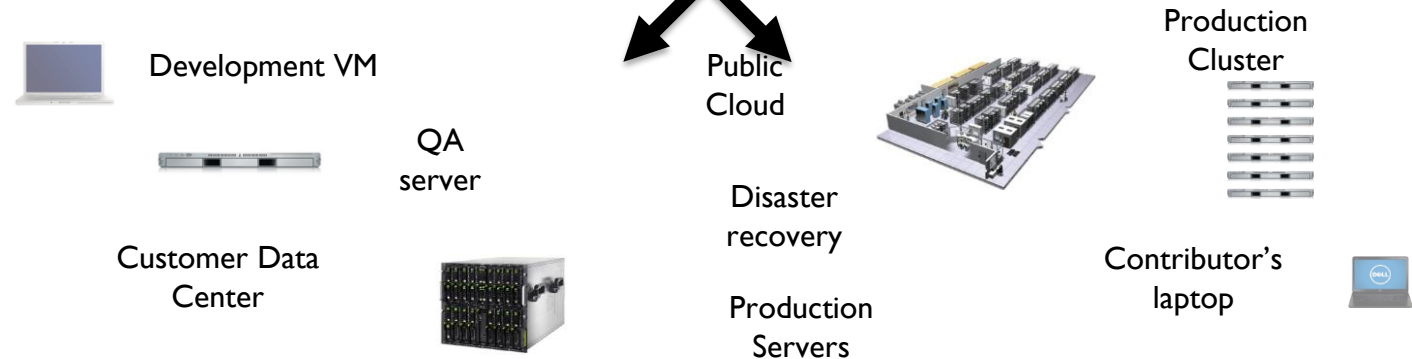
Multiplicity of
Stacks



Do services and
apps interact
appropriately?



Multiplicity of
hardware
environments



Can I migrate
smoothly and
quickly?

The Matrix From Hell

	Static website	?	?	?	?	?	?	?
	Web frontend	?	?	?	?	?	?	?
	Background workers	?	?	?	?	?	?	?
	User DB	?	?	?	?	?	?	?
	Analytics DB	?	?	?	?	?	?	?
	Queue	?	?	?	?	?	?	?
		Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
								

Cargo Transport Pre-1960

Multiplicity of
Goods



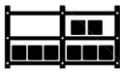
Do I worry about
how goods interact
(e.g. coffee beans
next to spices)

Multiplicity of
methods for
transporting/storing

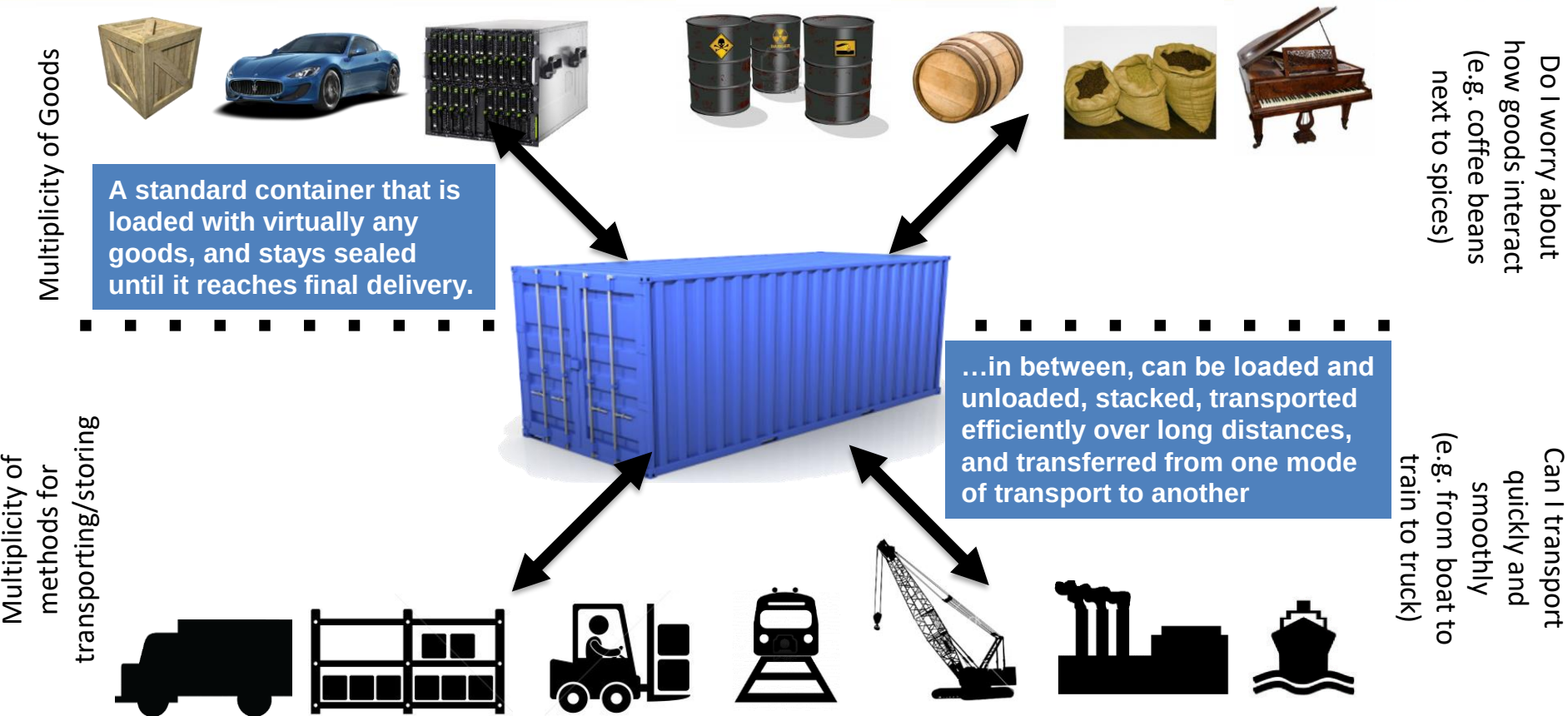


Can I transport
quickly and smoothly
(e.g. from boat to
train to truck)

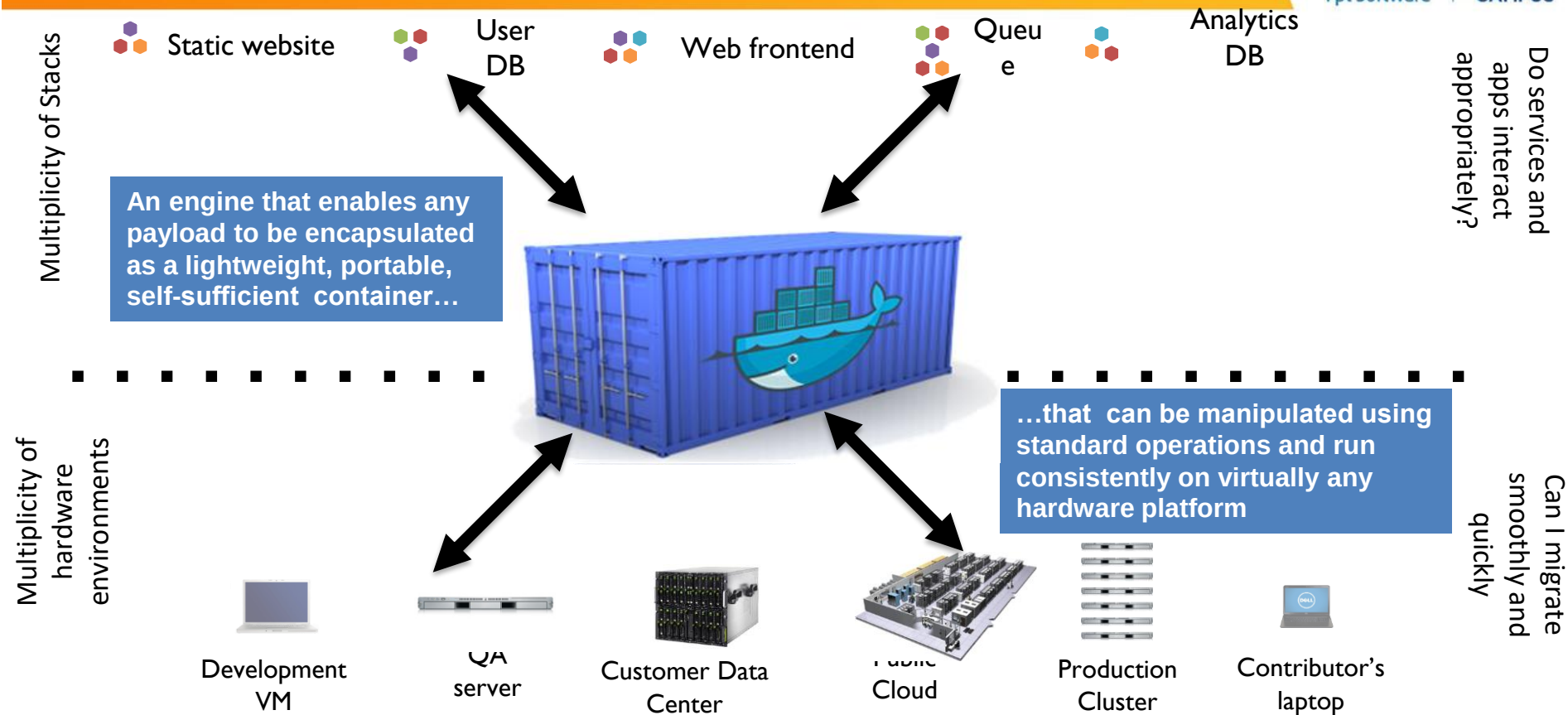
Also a matrix from hell

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

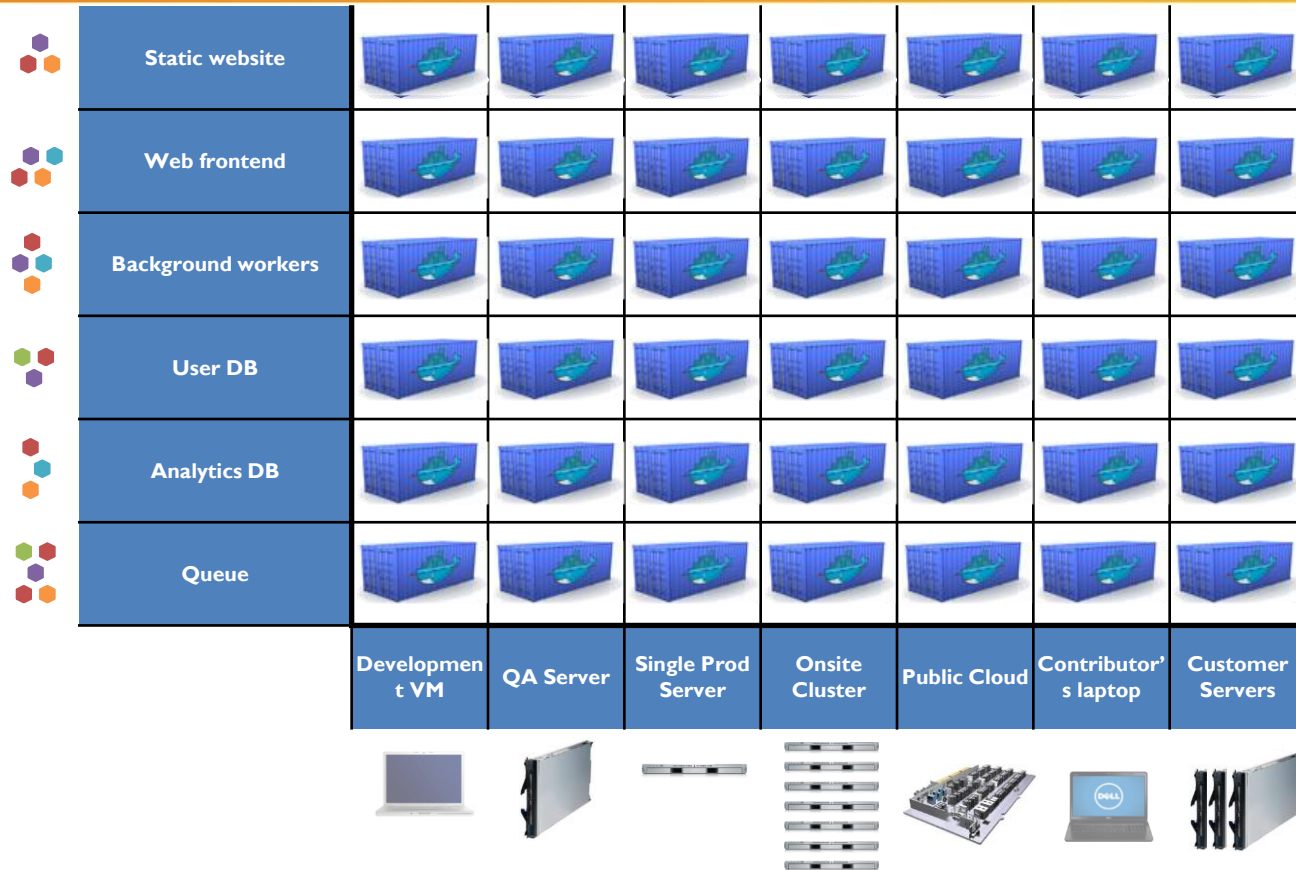
Solution: Intermodal Shipping Container



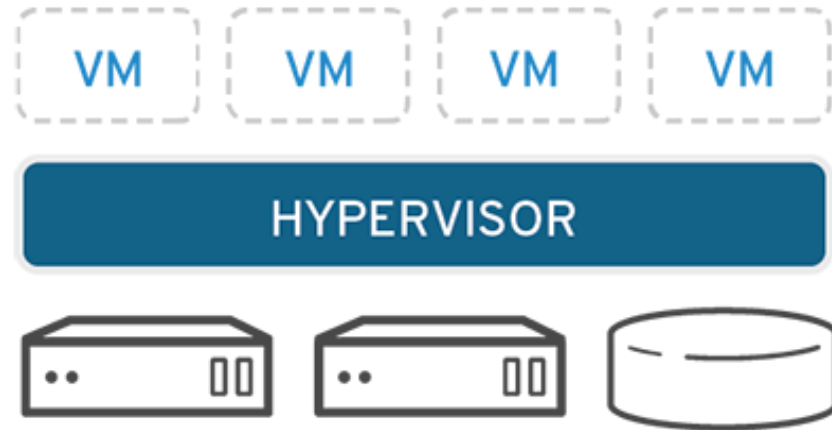
Docker is a shipping container system for code



Docker eliminates the matrix from Hell

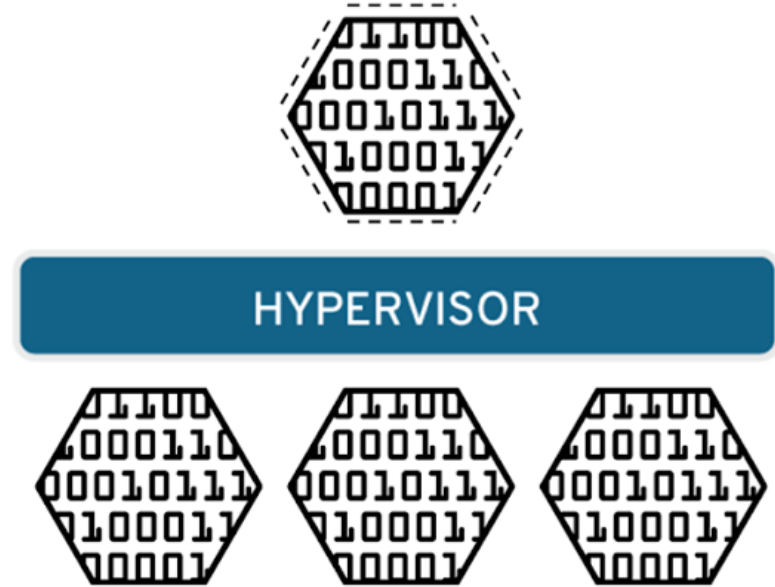


Virtualization is technology that lets you create useful IT services using resources that are traditionally bound to hardware. It allows you to use a physical machine's full capacity by distributing its capabilities among many users or environments.



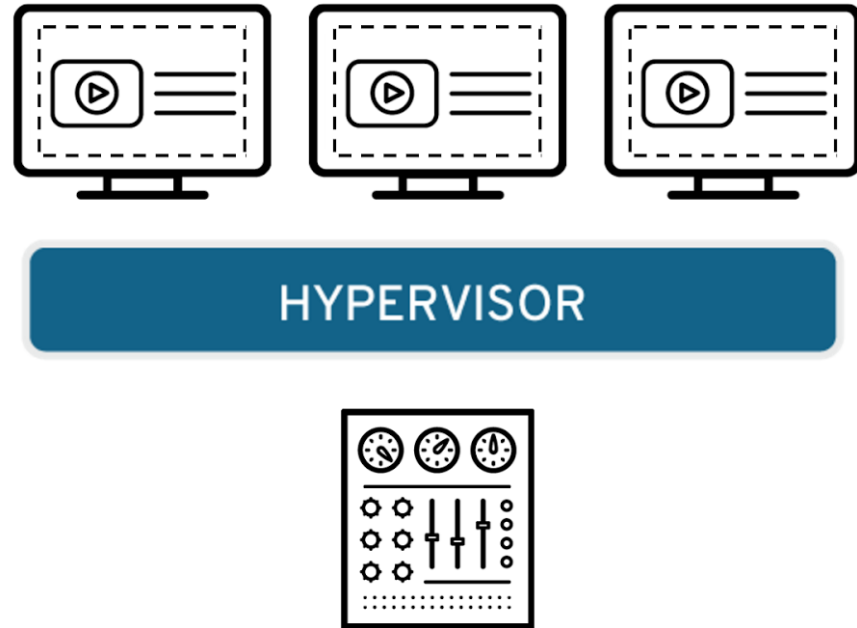
Data virtualization

Data that's spread all over can be consolidated into a single source. Data virtualization allows companies to treat data as a dynamic supply—providing processing capabilities that can bring together data from multiple sources, easily accommodate new data sources, and transform data according to user needs



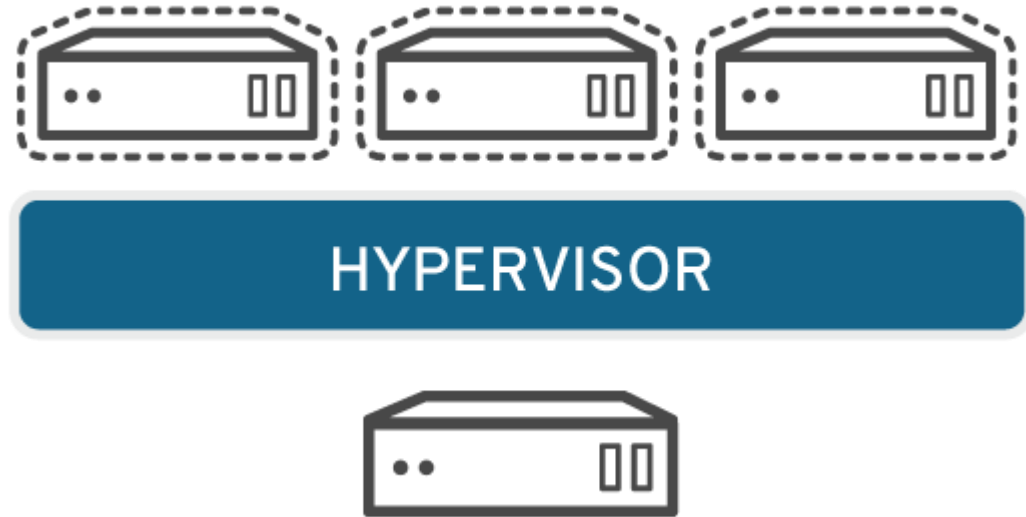
Desktop virtualization

Easily confused with operating system virtualization—which allows you to deploy multiple operating systems on a single machine—desktop virtualization allows a central administrator (or automated administration tool) to deploy simulated desktop environments to hundreds of physical machines at once



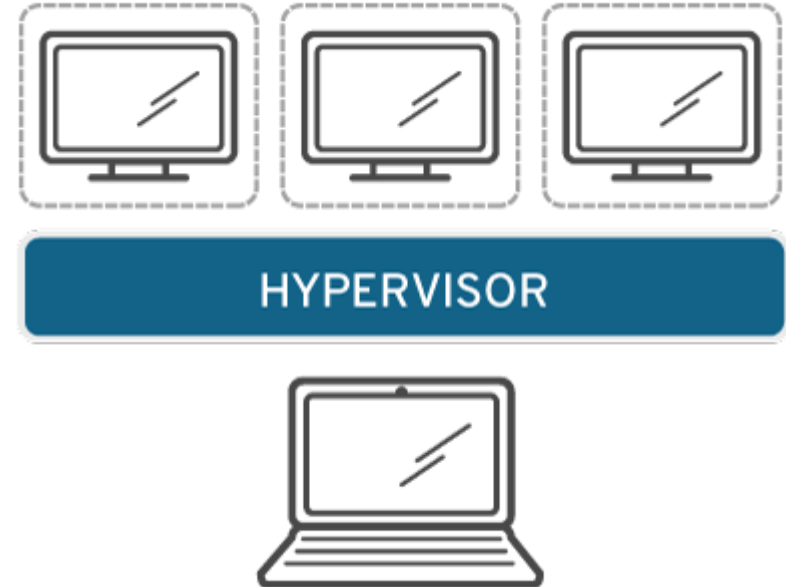
Server virtualization

Servers are computers designed to process a high volume of specific tasks really well so other computers—like laptops and desktops—can do a variety of other tasks. Virtualizing a server lets it to do more of those specific functions and involves partitioning it so that the components can be used to serve multiple functions.



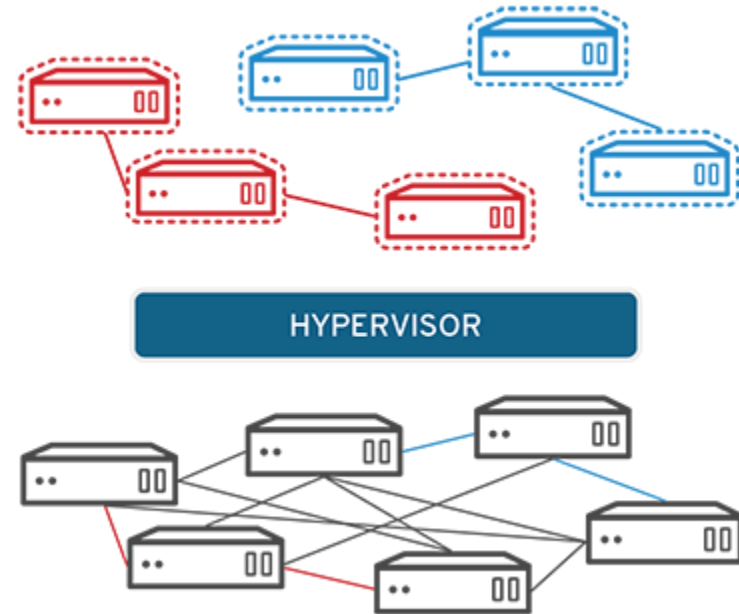
Operating system virtualization

Operating system virtualization happens at the kernel—the central task managers of operating systems. It's a useful way to run Linux and Windows environments side-by-side



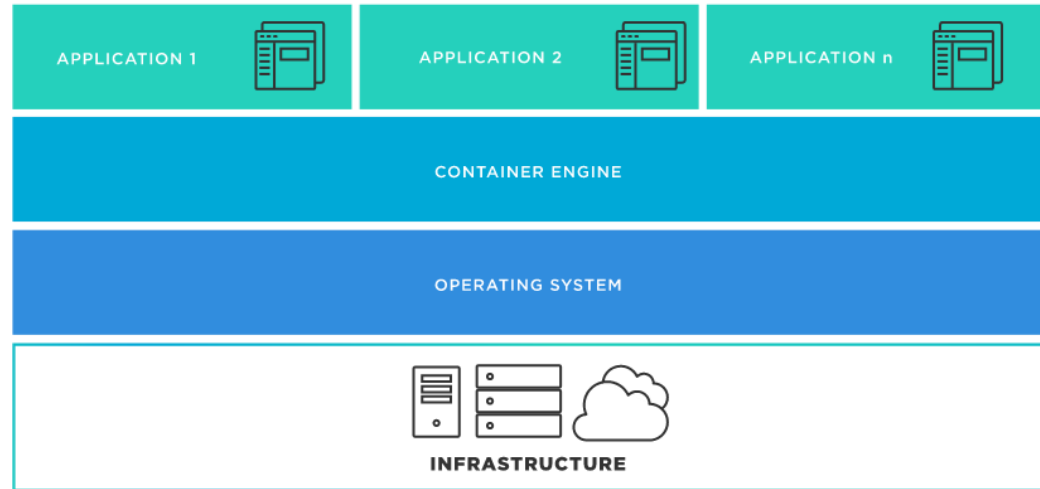
Network functions virtualization

Network functions virtualization (NFV) separates a network's key functions (like directory services, file sharing, and IP configuration) so they can be distributed among environments.

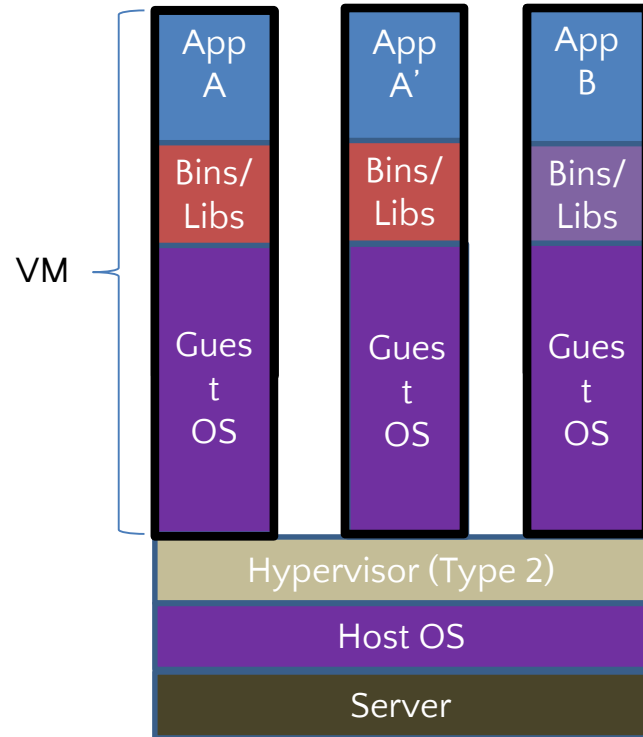


What is containerization?

Containerization is the packaging of software code with just the operating system (OS) libraries and dependencies required to run the code to create a single lightweight executable—called a container—that runs consistently on any infrastructure

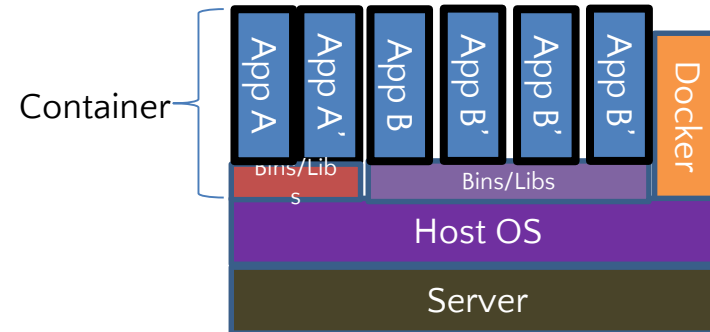


Containers vs. VMs



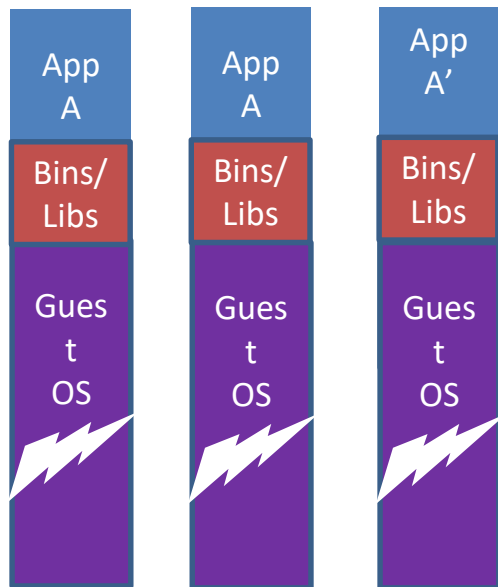
Containers are isolated,
but share OS and, where
appropriate,
bins/libraries

...result is significantly faster
deployment, much less overhead,
easier migration, faster restart



Why are Docker containers lightweight?

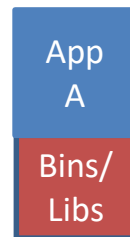
VMs



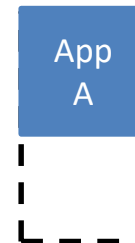
VMs

Every app, every copy of an app, and every slight modification of the app requires a new virtual server

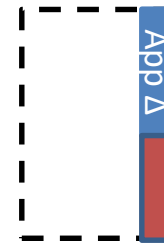
Containers



Original App
(No OS to take up space, resources, or require restart)



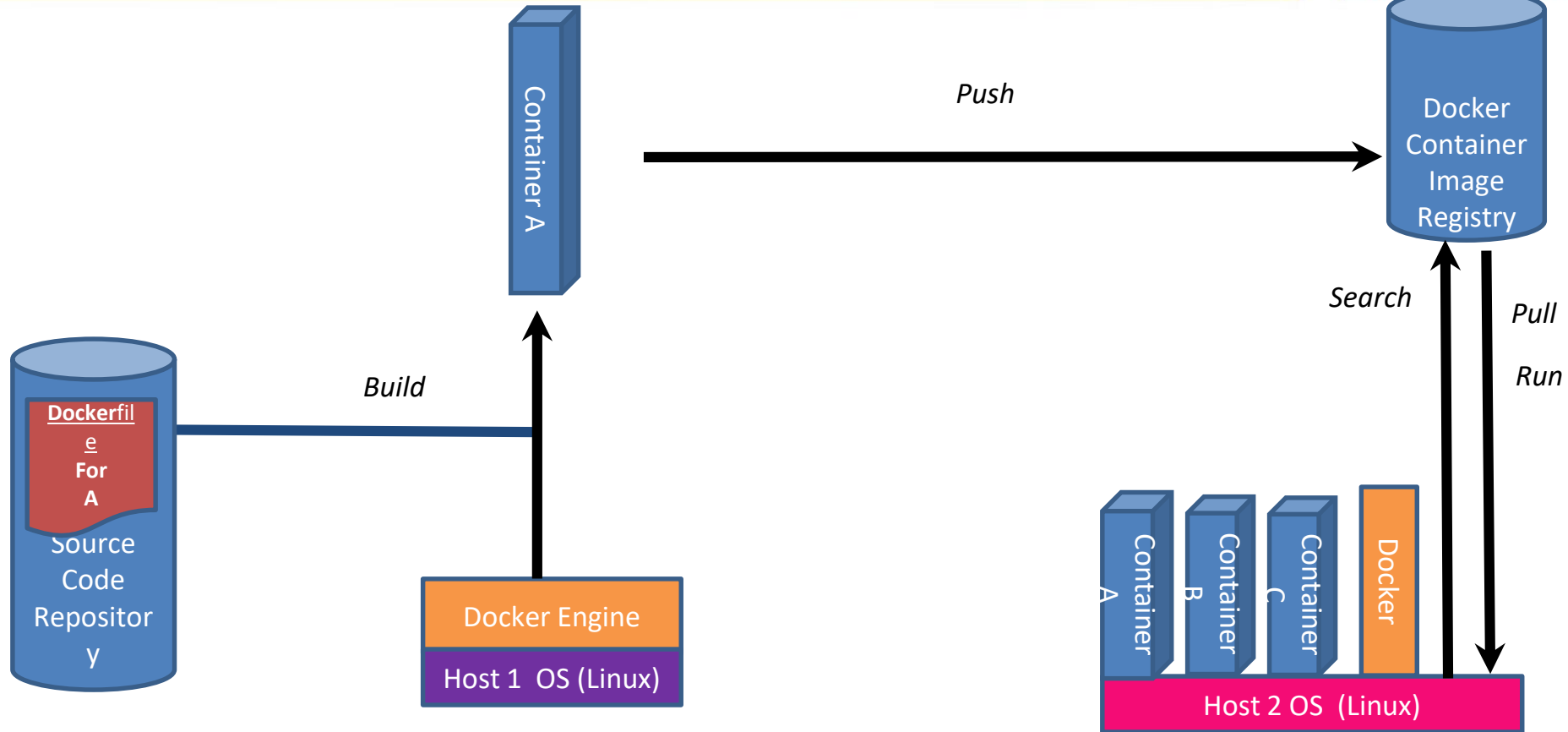
Copy of App
No OS. Can Share bins/libs



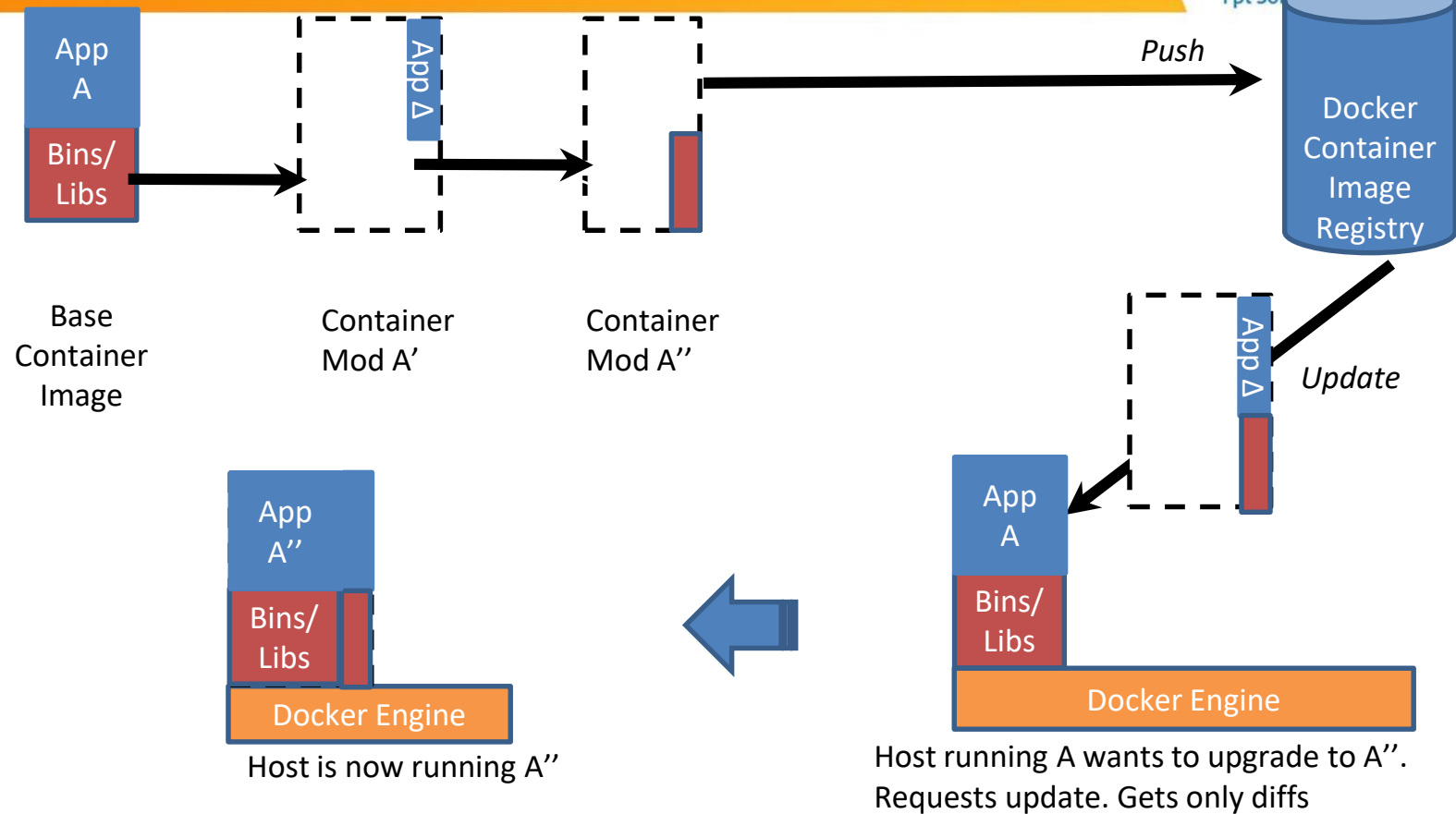
Modified App

Copy on write capabilities allow us to only save the diffs Between container A and container A'

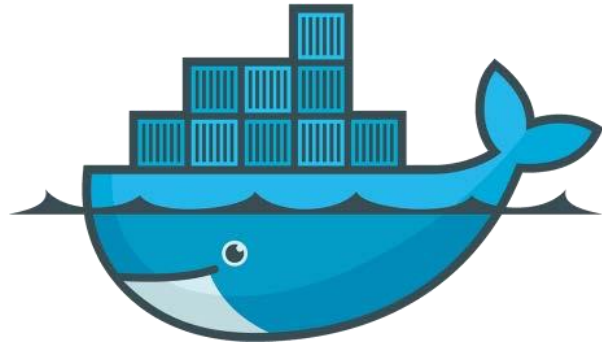
What are the basics of the Docker system?



Changes and Updates

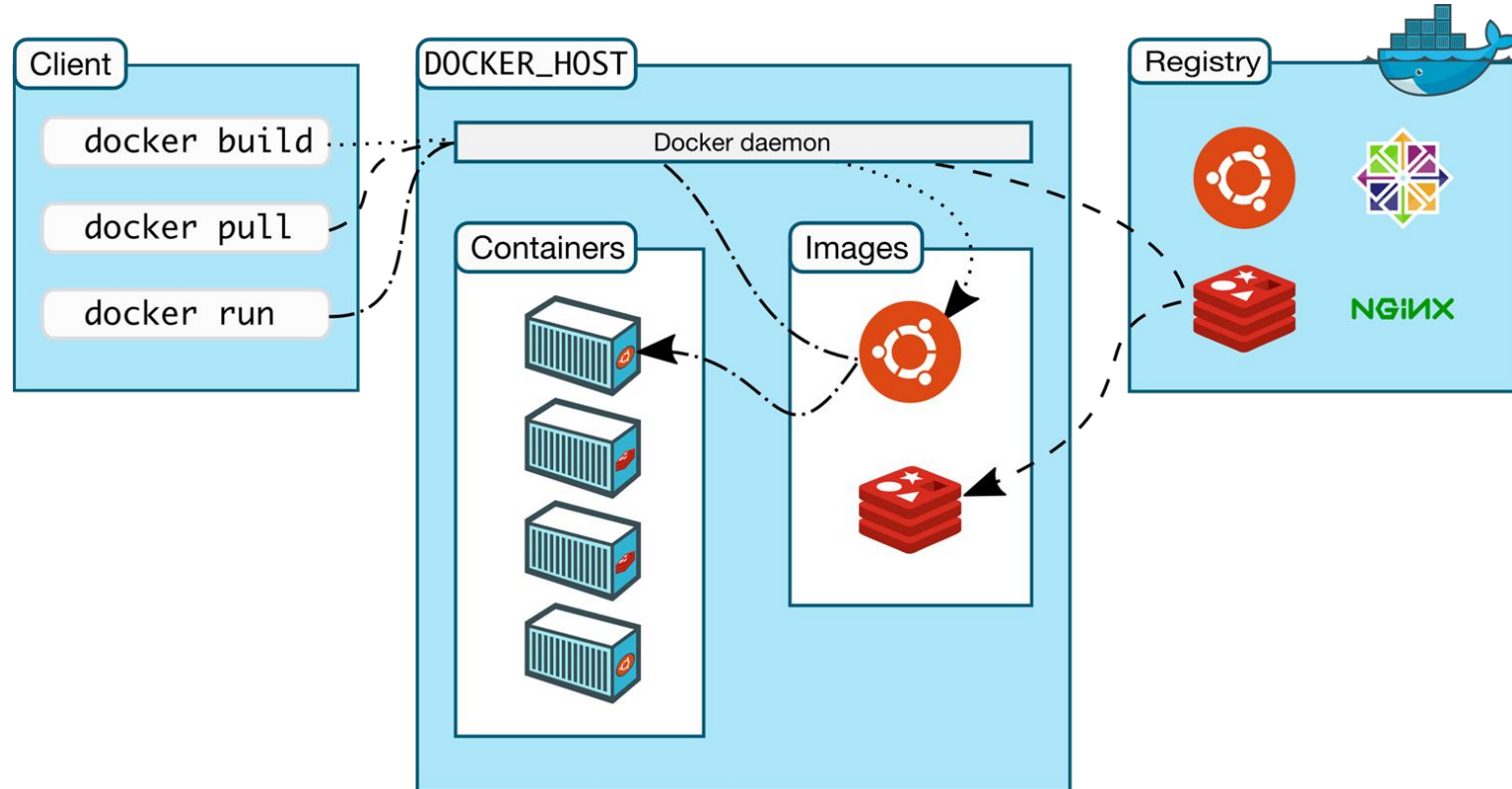


What Is Docker?



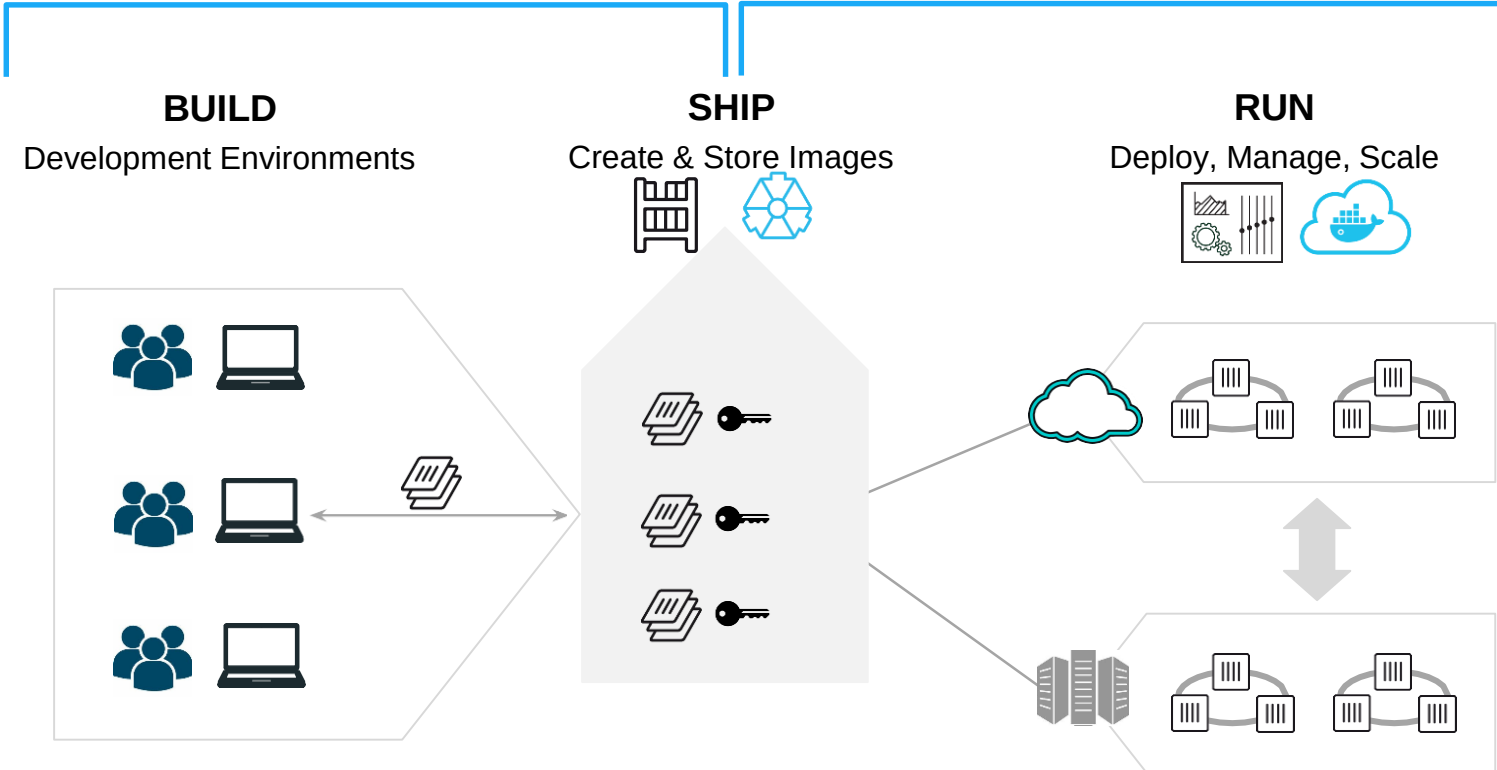
- **Lightweight, open, secure platform**
- **Simplify building, shipping, running apps**
- Runs natively on Linux or Windows Server
- Runs on Windows or Mac Development machines (with a virtual machine)
- Relies on "images" and "containers"

Docker architecture



Using Docker: Build, Ship, Run Workflow

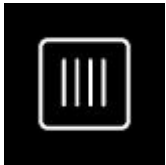
Developers IT Operations





Docker Image

The basis of a Docker container. Represents a full application



Docker Container

The standard unit in which the application service resides and executes



Docker Engine

Creates, ships and runs Docker containers deployable on a physical or virtual, host locally, in a datacenter or cloud service provider



Registry Service (Docker Hub(Public) or Docker Trusted Registry(Private))

Cloud or server based storage and distribution service for your images

Basic Docker Commands

```
$ docker image pull node:latest
```

```
$ docker image ls
```

```
$ docker container run -d -p 5000:5000 --name node node:latest
```

```
$ docker container ps
```

```
$ docker stop <container id>
```

```
$ docker rm <container id>
```

```
$ docker image rmi (or <image id>)
```

```
$ docker build -t node:2.0 .
```

```
$ docker image push node:2.0
```

```
$ docker --help
```

